

额外的存储容量来运行 Web 应用程序。

目前可通过两种通用方法构建 Web 应用程序：在服务器上执行大部分应用程序逻辑的传统 Web 应用程序，以及在 Web 浏览器中执行大部分用户界面逻辑的单页应用程序（Single Page Web Application, SPA），后者主要使用 Web API 与 Web 服务器通信。也可以将两种方法混合使用，最简单的方法是在更大型的传统 Web 应用程序中托管一个或多个丰富 SPA 类子应用程序。

16.1.2 Web应用特征

新式 Web 应用程序比以往承载着更高的用户期望和要求。当今的 Web 应用需要能在全球任何地方、在任意时刻可用，需要能在任何设备或屏幕尺寸上使用。Web 应用程序必须具有安全性、灵活性和可缩放性，以便满足高峰需求。如今复杂方案的处理日益需要丰富的用户体验，这建立在使用 JavaScript 并通过 Web API 进行有效通信的客户端的基础之上。新式 Web 应用程序具有以下特征。

1. 云托管和可扩展性

由于低内存、高吞吐量的特点，新式 Web 应用程序需要针对云（包括公共云、私有云以及任何云）进行优化。Web 应用程序占用空间更小，因此在相同硬件上可托管更多此类应用。并且，使用即用即付云托管服务时仅需为极少数资源付费。更高的高吞吐量意味着，通过相同硬件上的一个应用，Web 应用系统可服务更多客户，进而降低服务器和托管基础设施所需的投入。

2. 跨平台

大部分新式 Web 应用具有跨平台性，可在 Linux、macOS 以及 Windows 系统上运行。此功能为开发和部署的应用带来了新的选择。Docker 容器（包括 Linux 和 Windows）可托管 Web 应用程序，从而使应用程序可利用容器和微服务的优势。

3. 自动化测试

自动化测试能够利用自动化测试工具和框架，通过编写脚本来模拟用户在 Web 页面上的操作，自动化执行测试用例，比较预期结果和实际结果，包括一系列日志信息以及测试报告的自动生成，验证 Web 应用程序的正确性、稳定性和性能等指标。Web 应用自动化测试是一种高效、准确、可重复的测试方法，可以帮助开发人员和测试人员快速发现和解决 Web 应用程序中的缺陷和问题，同时也可以提高测试效率、降低测试成本、缩短测试周期。

4. 支持传统应用和 SPA

传统 Web 应用程序涉及极少的客户端行为，而是依赖服务器执行该应用可能需要执行的所有导航、查询和更新。用户执行的每个新操作会被转换为新的 Web 请求，其结果是最终用户浏览器中的完整页面重载。经典模型视图控制器（MVC）框架通常采用此方式，其中每个新请求对应一个不同的控制器操作，这些操作会反过来作用于模型并返回一个视图。特定页面上的一些单独操作可使用 AJAX（异步 JavaScript 和 XML）功能改进，但该应用的总体体系结构使用

了多个不同的 MVC 视图和 URL 终结点。此外，ASP.NET Core MVC 还支持 Razor Pages，这是一种组织 MVC 样式页的较为简单的方法。

与之相反，单页应用程序（SPAs）涉及极少动态生成的服务器端页面负载（如有）。许多 SPA 在一个静态 HTML 文件中进行初始化，此文件加载该应用启动和运行所需的 JavaScript 库。这些应用大量使用 Web API 处理数据需求，并提供更为丰富的用户体验。BlazorWebAssembly 提供了一种使用 .NET 代码生成 SPA 的方法，然后在客户端的浏览器中运行。

5. 简单的开发和部署

可使用简单的文本编辑器、命令行接口或者全功能开发环境（如 Visual Studio）编写 Web 应用程序。整体应用程序通常部署到单个终结点。可轻松将部署自动化，作为持续集成（CI）和持续交付（CD）管道的一部分。除传统的 CI/CD 工具外，Microsoft Azure 还集成了 git 存储库支持，并可在对特定 git 分支或标记作出更新时自动部署更新。Azure DevOps 提供了功能完备的 CI/CD 生成和部署管道，而 GitHub Actions 为在那里托管的项目提供了另一个选择。

16.2 Web 应用架构设计

16.2.1 Web应用架构设计原则

Web 应用架构的设计原则可以帮助开发人员作出体系结构决策，构建简洁、可维护的应用程序。一般而言，在这些原则的指导下构建的应用程序各部分间可通过显式接口或消息传送系统进行通信，它们并非松散耦合的离散组件。以下为 Web 应用架构的通用设计原则。

1. 分离关注点

分离关注点是开发时的指导原则。此原则主张根据软件执行的工作类型将软件分离。例如，假设应用程序中包含两个逻辑，其中一个逻辑标识要显示给用户的注意事项，另一个以特定方式设置这些注意事项的格式，使其更加显眼。负责选择为哪些事项设置格式的行为应与负责设置格式的行为区分开，因为这两种行为只是碰巧彼此相关联的独立关注点。

从体系结构上来说，按此原则有逻辑地构建应用程序应将核心业务行为与基础结构及用户界面逻辑区分开。理想情况下，业务规则和逻辑应单独位于一个项目中，且该项目不依赖应用程序中的其他项目。此区分操作有助于确保该业务模型易于测试，且可在不与低级别实现详细信息紧密耦合的情况下逐步改进（如果基础结构问题取决于业务层中定义的抽象时，这也有帮助）。在应用程序体系结构的使用层背后，关注点分离是核心设计思想。

2. 封装

应用程序的不同部分应通过封装与应用程序中的其他部分进行隔离。只要不违反外部协定，应用程序的组件和分层应该能够在不中断其协作者的情况下调整其内部实现。正确使用封装有助于在应用程序设计中实现松散耦合及模块化，因为只要维持相同的接口，就可以用替代实现来替代对象和包。

在类中实现封装的方式是限制对该类内部状态的外部访问权限。如果外部参与者想操作对象的状态，则应通过明确定义的函数（或属性 setter）来进行操作，而非直接访问该对象的私有状态。同样，应用程序组件和应用程序本身应公开明确定义的接口供协作者使用，而非让协作者直接修改其状态。通过此方法，只要公共协定得到维护，就可以不断改进应用程序的内部设计，而无须担心会中断协作者。

可变全局状态与封装是对立的。不能依赖从一个函数中的可变全局状态获取的值在另一个函数中（甚至在同一个函数中）具有相同的值。理解可变全局状态问题是 C# 语言等编程语言支持不同范围规则的原因之一，从语句到方法再到类，这些规则被广泛使用。值得注意的是，对于依赖中央数据库在应用程序内部和应用程序之间进行集成的数据驱动的体系结构，它们本身选择依赖由数据库表示的可变全局状态。域驱动的设计和整洁的体系结构的一个重要考虑因素是如何封装对数据的访问，以及如何确保应用程序状态不会因直接访问其持久性格式而无效。

3. 依赖关系反转

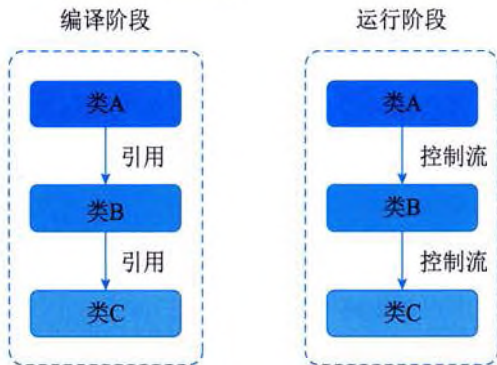


图 16-1 直接依赖关系图

应用程序中的依赖关系方向应该是抽象的方向，而不是实现详细信息的方向。大部分应用程序都是这样编写的：编译时依赖关系顺着运行时执行的方向流动，从而生成一个直接依赖项关系图。也就是说，如果类 A 调用类 B 的方法，类 B 调用 C 类的方法，则在编译时，类 A 将取决于类 B，而类 B 又取决于类 C，如图 16-1 所示。

应用依赖关系反转原则后，如图 16-2 所示，类 A 可以调用类 B 实现的抽象方法，让类 A 可以在运行时调用类 B，而类 B 又在编译时依赖于类 A 控制的接口（典型的编译时依赖项发生反转）。运行时，程序执行的流程保持不变，但接口引入意味着可以轻松插入这些接口的不同实现。

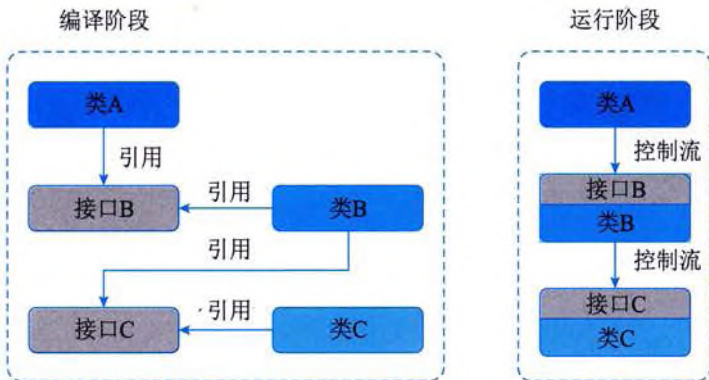


图 16-2 反转依赖关系图

依赖项反转是生成松散耦合应用程序的关键一环，因为可以将实现详细信息编写为依赖并

实现更高级别的抽象，而不是相反。因此，生成的应用程序的可测试性、模块化程度以及可维护性更高。遵循依赖关系反转原则可实现依赖关系注入。

4. 显式依赖关系

方法和类应显式要求正常工作所需的任何协作对象，一般将此称为显式依赖关系原则。通过类构造函数，类可以标识其实现有效状态和正常工作所需的内容。如果定义的类可供构造和调用，但仅在具备特定全局组件或基础结构组件时正常工作，则这些类对其客户端而言就不诚实。构造函数协定将告知客户端，它只需要指定的内容（如果类只使用无参数构造函数，则可能不需要任何内容），但随后在运行时，结果发现对象确实需要某些其他内容。

若遵循显式依赖关系原则，类和方法就会诚实地告知客户端其需要哪些内容才能工作。遵循此原则可以让代码更好地自我记录，并让代码协定更有利于用户，因为用户相信只要他们以方法或构造函数参数的形式提供所需的内容，他们使用的对象在运行时就能正常工作。

5. 单一责任

单一责任原则适用于面向对象的设计，但也可被视为类似于分离关注点的体系结构原则。它指出对象只应有一个责任，并且只能因为一个原因更改对象。具体而言，只在必须更新对象执行其唯一责任的方式时才更改对象。遵循这一原则有助于生成更松散耦合和模块化的系统，因为许多类型的新行为可以作为新类实现，而不是通过向现有类添加其他责任。添加新类始终比更改现有类安全，因为还没有任何代码依赖于新类。

在整体应用程序中，可以在高级别将单一责任原则应用于应用程序中的层。显示责任应位于UI项目中，而数据访问责任应位于基础结构项目中。业务逻辑应位于应用程序核心项目中，该项目易于测试，并且可以独立于其他责任进行逐步改进。

将此原则应用到应用程序体系结构及其逻辑终结点时，将获得微服务。给定的微服务应具有单一责任。一般而言，如果需要扩展系统的行为，最好通过添加其他微服务来实现，而不要向现有微服务添加责任。

6. 避免自我重复

应用程序应避免在多个位置指定与特定概念相关的行为，因为这种做法经常会导致出错。有时，如果要求发生变化，则要求更改此行为。至少有一个行为实例可能无法更新，系统行为可能不一致。

请将逻辑封装在编程构造中，而不要重复该逻辑。让此构造成为针对此行为的单一权限，并让应用程序中需要此行为的任何其他部分都使用新的构造。避免将恰巧重复的行为绑定在一起。例如，只因为两个不同的常数具有相同的值，如果从概念上讲，两个常数是指不同的内容，则并不意味着只应使用一个常数。复制总是比耦合到错误的抽象更可取。

7. 持久性无知

持久性无知是指需要保持不变的类型，但其代码不受所选择的持久性技术的影响。在.NET中的这种类型有时被称为普通旧CLR对象（Plain Old CLR Object, POJO），因为这种类型无须继承特定的基类或实现特定的接口。持久性无感知非常有用，因为它可以让相